Painel de Concursos

Documentação técnica e guia de implantação no GitHub

React 18 · Vite 5 · Tailwind CSS 3.4 · 100% no navegador

Sobre este documento

Este documento acompanha o arquivo **painel-concursos.zip**, que contém o código-fonte completo para publicar o “Painel de Concursos” em um repositório do GitHub. Ele explica o que o painel faz, como o código está organizado, como rodar localmente, como publicar no GitHub Pages e como os dados são guardados.

O projeto reproduz o artefato publicado no Claude. A reprodução foi conferida contra o original (veja a seção 9).

Sumário

- 1. Visão geral
- 2. Início rápido
- 3. Estrutura do repositório
- 4. Arquitetura da aplicação
- 5. Regras de negócio implementadas
- 6. Dados e persistência
- 7. Publicação no GitHub Pages
- 8. Personalização
- 9. Verificação de fidelidade e origem do código
- 10. Solução de problemas

1. Visão geral

O Painel de Concursos é um painel operacional para organizar a preparação para concursos públicos. Ele transforma o edital em uma matriz de assuntos, calcula prioridades, monta o plano de estudo, agenda revisões espaçadas e mostra o desempenho em questões e simulados.

Ele **não tem servidor nem banco de dados**. Todo o processamento acontece no navegador e os dados ficam no `localStorage`, com exportação e importação em JSON. Por isso pode ser hospedado como site estático (GitHub Pages, Netlify, Nginx, etc.).

Rota	Tela	Função
#hoje	Hoje	Próxima melhor ação, o que estudar agora, atrasos e riscos, evolução do desempenho
#concursos	Concursos	Comparar concursos, interseção de assuntos; fatos, prazos, fontes e pendências
#matriz	Matriz	Hierarquia de assuntos, prioridades, diagnóstico inicial, importação em massa
#cronograma	Cronograma	Abas Semana, Mês, Ciclo e Backlog; plano automático e replanejamento
#revisoes	Revisões	Revisões vencidas e próximas (14 dias), retenção, registro de resultado
#questoes	Questões	Registro, caderno de erros, calibração (confiança × acerto), lote de questões
#simulados	Simulados	Lançamento por bloco, pontuação, evolução e principais lacunas
#notebook	NotebookLM	Registros por assunto e prompts prontos para copiar
#relatorio	Relatório	Resumo semanal com recomendações e nível de evidência
#config	Configurações	Disponibilidade, bloqueios, metas, pesos, tema, backup JSON e CSV

Stack. React 18.3.1, Vite 5, Tailwind CSS 3.4.17 (com cores em variáveis CSS para tema claro/escuro), lucide-react 0.469 para ícones. Gráficos em SVG e CSS puros, sem biblioteca de gráficos.

2. Início rápido

Requisitos: **Node.js 18.18 ou superior** (recomendado 20) e npm. O arquivo `.nvmrc` indica a versão 20.

```
unzip painel-concursos.zip
cd painel-concursos
npm install        # instala as dependências
npm run dev        # abre em http://localhost:5173
npm run build      # gera a pasta dist/ (site estático)
npm run preview    # serve o build para conferência
npm run format     # formata o código (Prettier)
```

O comando `npm run build` produz três arquivos em `dist/` (HTML, um CSS de cerca de 19 kB e um JS de cerca de 339 kB, 104 kB com gzip). Qualquer servidor de arquivos estáticos serve o resultado.

> **Importante:** a fonte IBM Plex Sans é carregada do Google Fonts (`index.html`). Sem internet o painel funciona normalmente, usando a fonte padrão do sistema.

3. Estrutura do repositório

```
painel-concursos/
├── index.html                  entrada HTML
├── package.json / package-lock.json
├── vite.config.js              base "./" (caminhos relativos)
├── tailwind.config.js          cores mapeadas para variáveis CSS
├── postcss.config.js
├── README.md
├── docs/Painel-de-Concursos-Documentacao.pdf
├── public/favicon.svg
├── .github/workflows/deploy.yml   publicação automática no GitHub Pages
└── src/
    ├── main.jsx                monta o React em #root
    ├── App.jsx                 estado global, persistência, rotas, menu
    ├── index.css               tokens de cor, Tailwind, ajustes globais
    ├── context/AppContext.js   contexto e hook useApp()
    ├── data/                   constants.js · defaults.js · storage.js
    ├── lib/                    dates · stats · planner · backup · clipboard · cn
    ├── hooks/                  useElementWidth · useSortableTable
    ├── components/
    │   ├── ui/                 Badge Button Card Modal form feedback table
    │   ├── charts/             LineChart · BarList · PlannedDoneChart
    │   └── SessionModal.jsx
    └── screens/                Today Concursos Matrix Schedule Reviews
                                Questions Simulados Notebook Report Settings
```

Em `src/` são 34 módulos (cerca de 9,4 mil linhas já formatadas). A dependência vai sempre de cima para baixo (`screens` → `components/hooks` → `lib` → `data`) e foi verificado que não existem importações circulares.

4. Arquitetura da aplicação

Estado único. O `App.jsx` guarda um único objeto de estado com as coleções `concursos`, `topics`, `sessions`, `questions`, `simulados`, `notebooks`, `decisions` e o bloco `settings`. As telas leem e alteram esse estado pelo hook `useApp()`, que expõe:

Função	Para que serve
state / setState	Estado completo e atualização por função
upsert(coleção, item) / remove(coleção, id)	Inserir-ou-atualizar e remover um item de uma coleção
patchSettings(parte)	Atualiza configurações (ao mexer em horasDia, tira o aviso “provisório”)
log(motivo, ação)	Registra uma decisão do planejamento em `decisions`
replaceAll(estado)	Substitui tudo (usado na importação de backup)
nav(rota) / openSession(sessão)	Navega pelo hash e abre o modal de sessão
sync	Modo de armazenamento atual (db, local ou memory) e última gravação

Roteamento por hash. A rota é o texto depois do # na URL (`#hoje`, `#matriz`…). Rota desconhecida cai em `#hoje`. Como não usa o histórico do navegador, o site funciona em qualquer hospedagem estática e em subpastas, sem regras de redirecionamento.

Tema. A configuração `tema` aceita *system*, *light* e *dark*. Em “system” segue o tema do sistema; nos outros casos o app define o atributo `data-theme` no elemento raiz. Todas as cores vêm das variáveis CSS de `src/index.css`.

Componentes de interface. Ficam em `components/ui` e usam classes do Tailwind. Os tons semânticos são *good, warn, crit, serious, info, muted, accent*.

Módulos de lógica. Não dependem de React e são funções puras sobre o estado:

Módulo	Conteúdo
lib/dates.js	Datas em ISO local, semana começando na segunda, meses, formatação de minutos, `uid()`
lib/stats.js	Contagem de acertos/erros/brancos, intervalo de Wilson, desempenho por assunto, tendência (inclinação linear), desvio-padrão, pontuação de simulado
lib/planner.js	Prioridades, classificação de domínio, revisão espaçada, plano diário, replanejamento de atrasos, alertas, próxima melhor ação, resumo semanal
data/defaults.js	Dados iniciais e fábricas de novos registros (concurso, assunto, questão, linha de simulado, anotação)
data/storage.js	Leitura/gravação no `localStorage` e sincronização opcional com o banco do host

5. Regras de negócio implementadas

Resumo do que o código faz hoje. Os valores padrão podem ser alterados em Configurações.

Prioridade dos assuntos

Cada assunto recebe uma nota combinando oito fatores, cada um com um peso configurável (padrão entre parênteses): relevância (3), peso na prova (2), lacuna de desempenho (3), facilidade (1), tempo (1,5), esquecimento (1,5), número de concursos que cobram o assunto (1,5) e proximidade da prova (2,5). A tela Matriz permite ordenar por nota ou por nota por hora.

Estágio de preparo

Com a opção “auto”, o estágio é calculado pela prova mais próxima entre os concursos ativos com edital publicado: mais de 90 dias = **Base**; entre 31 e 90 = **Consolidação**; 30 dias ou menos = **Reta final**. Cada estágio define a divisão do tempo entre estudo profundo, consolidação e avaliação:

Estágio	Estudo profundo	Consolidação	Avaliação
Base	60%	30%	10%
Consolidação	40%	45%	15%
Reta final	20%	45%	35%

Disponibilidade líquida

Por dia da semana há uma disponibilidade em minutos (padrão: domingo 180, segunda a sexta 120, sábado 240). O tempo líquido do dia é `(disponibilidade − bloqueios do dia) × (1 − folga%)`, com folga padrão de 15%. Sessões padrão têm 50 minutos, revisões 25, e no máximo 3 assuntos por dia.

Revisão espaçada

Os intervalos iniciais são 1, 7 e 30 dias. O próximo intervalo é ajustado pela relevância e pela dificuldade do assunto, pelo acerto em questões (acima de 85% alonga, abaixo de 60% encurta, com amostra mínima de 10 questões) e pelo resultado da revisão (lembrei bem, em parte, esqueci). Revisões nascem quando uma teoria é marcada como concluída.

Evidência e amostra pequena

Percentuais de acerto vêm com intervalo de Wilson e são marcados como indicativos quando a amostra é pequena. O relatório semanal classifica cada recomendação como “evidência suficiente”, “indicativo” ou “dados insuficientes”.

Simulados

A pontuação segue a regra configurada no concurso (por exemplo, cada erro anula um acerto) e compara cada bloco com a nota mínima. Os lançamentos alimentam as estatísticas por assunto.

6. Dados e persistência

Item	Detalhe
Onde ficam	`localStorage` do navegador, chave `prep-concursos-v1`
Quando grava	Automaticamente, 400 ms após cada alteração
Formato	JSON com `v: 1`, `settings, concursos, topics, sessions, questions, simulados, notebooks, decisions`
Backup	Configurações → Exportar JSON (backup completo); Importar JSON valida o arquivo antes de substituir tudo
CSV	Exportação de questões, com BOM para abrir direto no Excel
Dados iniciais	Definidos em `src/data/defaults.js`: 4 concursos (CGU, SEFAZ-DF, SES-DF, BACEN), 13 assuntos do CGU e configurações padrão

Cuidado. Limpar os dados do navegador, usar janela anônima ou trocar de navegador/computador significa começar do zero. Cada pessoa tem os próprios dados. Exporte backups com frequência.

Integração opcional com o Claude

Se existir `window.claude` (app publicado como artefato no Claude), o painel usa o banco compartilhado (`db`) e o download assistido (`downloads`). Fora dali essas chamadas não existem: o app usa `localStorage` e o download normal do navegador, sem nenhuma configuração.

7. Publicação no GitHub Pages

O workflow `.github/workflows/deploy.yml` já está pronto: instala as dependências com `npm ci`, roda o build e publica a pasta `dist/`.

Passo a passo

1. Crie um repositório novo no GitHub (por exemplo **painel-concursos**).
2. No computador, dentro da pasta do projeto, envie o código:

```
git init
git add .
git commit -m "Painel de Concursos"
git branch -M main
git remote add origin https://github.com/SEU-USUARIO/painel-concursos.git
git push -u origin main
```

3. No repositório, abra **Settings → Pages** e, em **Build and deployment → Source**, escolha **GitHub Actions**.
4. Abra a aba **Actions** e aguarde o workflow "Deploy para o GitHub Pages" terminar (cerca de 1 minuto).
5. O site fica em `https://SEU-USUARIO.github.io/painel-concursos/`. Cada novo `git push` na `main` republica.

Por que não precisa configurar o nome do repositório? O build usa `base: "./"` (caminhos relativos) e o roteamento é por hash. Se quiser caminhos absolutos, defina a variável `VITE_BASE=/painel-concursos/` no passo de build.

Outras hospedagens

Netlify, Vercel, Cloudflare Pages ou Nginx: comando de build `npm run build` e pasta de saída `dist`. Não é preciso nenhuma regra de reescrita de URL.

> **Privacidade.** Os dados de estudo ficam só no navegador de quem usa e não vão para o repositório. Se você editar `src/data/defaults.js` com informações pessoais, use um repositório privado. O GitHub Pages em repositório privado exige plano compatível.

8. Personalização

Quero…	Onde mexer
Trocar cores ou o tema	`src/index.css`: variáveis `--accent`, `--surface`, `--ink`… (claro e escuro). O Tailwind apenas aponta para elas.
Começar com outros concursos/assuntos	`src/data/defaults.js`: `defaultConcursos()`, `defaultTopics()`, `defaultSettings()`. Só vale para quem ainda não tem dados salvos no navegador.
Mudar metas, pesos, disponibilidade	Tela Configurações, ou os padrões em `defaultSettings()`
Alterar rótulos e listas fixas	`src/data/constants.js`
Mudar a fórmula de prioridade ou de revisão	`src/lib/planner.js` (`computePriorities`, `computeReview`)
Criar uma nova tela	Criar o componente em `src/screens/` e adicionar uma linha em `NAV_ITEMS` no `App.jsx`
Tirar menções a "artifact" e "Claude" do texto	`src/screens/SettingsScreen.jsx` e `src/screens/ReportScreen.jsx`

Formatação: `npm run format` aplica o Prettier (largura 100, aspas duplas).

9. Verificação de fidelidade e origem do código

Origem. O artefato foi publicado já compilado (bundle minificado, com o CSS do Tailwind gerado). O projeto foi reconstruído a partir dele: os módulos, componentes, funções, constantes e props foram renomeados para nomes legíveis (por exemplo `TodayScreen`, `computePriorities`, `STUDY_STAGES`), o JSX foi recuperado e o código foi dividido em 34 módulos. Algumas variáveis locais curtas (`a, s, t…`) permanecem dentro de funções, porque o nome original se perdeu na compilação.

Testes feitos (navegador real, Chromium, original × projeto novo):

Teste	Resultado
Texto renderizado das 10 telas	Idêntico nas 10
Capturas de tela, tema claro (1280 px)	0 pixels de diferença nas 10 telas
Capturas de tela, tema escuro e mobile (390 px)	0 pixels de diferença (Hoje, Matriz, Relatório, Configurações)
Clique em até 30 botões por tela (96 cliques)	Mesmo resultado em todos; nenhum erro de JavaScript
Exportar JSON e CSV, importar backup	Download real funcionando; diálogo de importação abre
`npm ci` **+ build em cópia limpa, com e sem** `VITE_BASE`	Build concluído nos dois casos

Diferenças intencionais em relação ao artefato:

- **Download real.** Fora do Claude, Exportar JSON e CSV baixam o arquivo pelo navegador. No artefato, sem a capacidade de download, o app mostrava o texto para copiar.
- **Build próprio.** React e ícones vêm do npm em vez de CDN, e o CSS é gerado pelo Tailwind do projeto (resultado visual idêntico).

Limite do que foi testado: a comparação foi feita com os dados iniciais e com cliques automatizados. Fluxos longos de preenchimento (por exemplo, meses de uso com muitas sessões) não foram reproduzidos um a um.

10. Solução de problemas

Sintoma	Causa provável e solução
Página em branco no GitHub Pages	Em Settings → Pages, a fonte precisa ser **GitHub Actions** (não "Deploy from a branch"). Confira na aba Actions se o workflow terminou sem erro.
Workflow falha em `npm ci`	O `package-lock.json` precisa estar no repositório. Não o apague nem o coloque no `.gitignore`.
Os dados sumiram	O `localStorage` é por navegador e por endereço. Outro navegador, janela anônima ou outro domínio começam vazios. Restaure por Configurações → Importar JSON.
Aviso ao fechar a aba	O app avisa quando não há armazenamento disponível e os dados só existem na memória. Exporte um backup antes de sair.
Fonte diferente do esperado	A IBM Plex Sans vem do Google Fonts. Sem internet ou com bloqueio, entra a fonte do sistema. Para hospedar a fonte, instale `@fontsource/ibm-plex-sans` e importe-a em `main.jsx`.
`npm run dev` **não abre**	Verifique a versão do Node (`node -v`, mínimo 18.18) e a porta 5173 livre.